

Project Proposal — Chess960WZ

A Two-Player Chess960 Game with Advanced Analytics Dashboard Version: 2.1 | Date: 6 April 2026 | Author: Phasathat Jaruchitsophon 6610545375

1. Project Overview

Chess960WZ is a local two-player chess game built with Python and Pygame, implementing the Chess960 (Fischer Random Chess) variant. Instead of a fixed starting position, the back-rank pieces are randomly arranged each game following Chess960 rules — creating 960 possible starting positions. This removes memorized openings and forces both players to think from move one.

The game records every move and exports a standard PGN file compatible with Lichess and Chess.com. A live algebraic notation panel updates beside the board in real time. After each session, an advanced statistics dashboard shows win rates, piece usage, move-time patterns, and tactical analysis — all computed from real recorded data using Pandas and Matplotlib.

Key features:

- Chess960 random position generator — new starting setup each game
 - Full standard chess rules — Chess960-aware castling, en passant, pawn promotion, check, checkmate
 - Four time control modes — Bullet (1 min), Blitz (3 min), Rapid (10 min), Classical (30 min)
 - Real-time algebraic notation panel — updates beside the board on every move
 - PGN export after game end — saved to /pgn_files/, loadable in any chess tool
 - Advanced analytics dashboard — 7 chart types with dynamic White/Black/All filter
 - Board flip with F key
 - Visual helpers — valid move highlights, check warning, last move indicator
-

2. Project Review

2.1 Reference Project

The reference project is the original Chess Master — a Python/Pygame chess game covering core gameplay, move validation, check detection, castling, en passant, pawn promotion, basic CSV statistics, and matplotlib charts.

2.2 Key Limitations of Original

- No Chess960 support — fixed starting position only
- No PGN export — games cannot be saved or shared externally
- No live move notation during gameplay
- Piece usage chart uses estimated percentages instead of real recorded data
- No opening detection or opening frequency analysis
- OOP design: all piece logic in one class, no Square class, no inheritance
- Heatmap misused — applied to a single column, not as relationship visualization

2.3 Improvements in Chess960WZ

- Chess960 random position generator using combinatorial algorithm ($4 \times 4 \times 6 \times 10 \times 1 = 960$ positions)
- Square class added — board built as 8x8 grid of Square objects
- Piece inheritance — ChessPiece as base class, King/Queen/Rook/Bishop/Knight/Pawn as subclasses
- PGNExporter class generates valid .pgn file after every game
- Real-time algebraic notation panel updates after every move
- Piece usage and capture statistics read from real moves_detail.csv data
- Statistics collected each turn — useful data from a single game, no need to play 100 games first
- Heatmap replaced with Lethality Matrix (attacker × victim stacked bar chart)
- New analytics: Capture Hesitation, Think Time by Phase, Move-Time Trend
- Dynamic chart filter — switch between All / White / Black views instantly
- Fully rebranded as Chess960WZ

3. Programming Development

3.1 Game Concept

Chess960WZ is a two-player local chess game on one machine. At game start, the back-rank pieces are randomly arranged following Chess960 rules. Pawns remain on the second rank.

Standard chess rules apply for all moves. A notation panel on the right updates live as moves are played.

Key features:

- Chess960 random position generator — new starting setup each game
- Full standard chess rules — Chess960-aware castling, en passant, pawn promotion, check, checkmate
- Four time control modes — Bullet (1 min), Blitz (3 min), Rapid (10 min), Classical (30 min)
- Real-time move notation panel during gameplay
- PGN export after game end — saved to /pgn_files/, loadable in any chess tool
- Statistics dashboard — charts generated from real per-turn data
- Board flip with F key
- Visual helpers — valid move highlights, check warning, last move indicator

3.2 Object-Oriented Programming Design

OOP structure follows TA feedback: the game contains a board, the board contains squares, the board has pieces, and pieces use inheritance.

Class	Role
ChessGame	Main controller — game loop, screen routing, turn logic, timer, stats_saved flag
ChessBoard	Contains 8x8 grid of Square objects, manages rendering, notation panel, and board-level logic
Square	Represents one cell — stores position, color, occupying piece
ChessPiece (base)	Abstract base class for all pieces — shared attributes and interface
King / Queen / Rook / Bishop / Knight / Pawn	Subclasses of ChessPiece — each implements its own get_valid_moves()
Chess960Generator	Generates valid Chess960 starting positions using combinatorial algorithm
ChessStatistics	Records move data per turn, stores to CSV using Pandas, generates 7 chart types
PGNExporter	Builds and exports standard PGN file from recorded moves

3.3 Algorithms

Chess960 Position Generation — 5 steps:

1. Place dark-squared bishop on one of 4 dark squares → 4 choices
2. Place light-squared bishop on one of 4 light squares → 4 choices
3. Place queen on one of 6 remaining squares → 6 choices
4. Place 2 knights on 2 of 5 remaining squares → $C(5,2) = 10$ choices
5. Place rooks and king in the 3 remaining squares — king must be between rooks → 1 arrangement

Total: $4 \times 4 \times 6 \times 10 \times 1 = 960$ unique positions

Other algorithms:

- Move validation — rule-based logic per piece subclass, checks boundaries and blocking pieces
 - Check and checkmate detection — scans all opponent moves after every turn; simulates escapes for checkmate
 - Chess960-aware castling — king travels to g1/c1, rook to f1/d1, path-clear and not-in-check verified along actual travel path
 - En passant tracking — records target square after pawn double-step, cleared next turn if unused
 - Algebraic notation — converts board coordinates to standard notation (Nf3, exd5, O-O, O-O-O)
 - PGN builder — formats headers (Event, Date, White, Black, Result, Variant, FEN) and move list with correct line wrapping
-

4. Statistical Data

4.1 Data Features

All statistics are computed from real per-move data written to CSV every turn. A single game of ~40 moves yields 40 rows in moves_detail.csv — enough to display meaningful charts immediately, without needing 100 games first.

	Why it is good to have this data? What can it be used for	How will you obtain 100 values of this feature data?	Which variable (and which class will you collect this from)?	How will you display this feature data (via summarization statistics or via graph)?
Feature 1: Win/Loss/Draw outcome	Shows overall game balance between White and Black. Can be used to analyze if one side has an advantage or if draws are common across sessions	Recorded automatically at game end. 100 rows = 100 completed games	winner column in games_history.csv — collected by ChessGame class	Pie chart + Summary table (count and % per outcome)
Feature 2: Game duration (sec)	Shows how long each game lasts. Can be used to compare playstyles (aggressive vs slow) and see how time control mode affects game length	Calculated as white_time_used + black_time_used at game end. 100 rows = 100 games	duration column in games_history.csv — collected by ChessGame class	Histogram + Summary table (Min, Max, Mean, Median, Std Dev)

Feature 3: Move count per game	Shows how complex or long each game is. Can be used to track whether games are getting longer over time or to compare game length across time controls	Total half-moves counted per game, recorded at game end. 100 rows = 100 games	<code>total_moves</code> column in <code>games_history.csv</code> — collected by <code>ChessGame</code> class	Line graph + Summary table (Min, Max, Mean, Median, Std Dev)
Feature 4: Piece type per move	Shows which piece types are moved most often. Can be used to reveal playstyle patterns — e.g., pawn-heavy vs knight-heavy openings	Recorded every turn as one row per move. 100 rows = approximately 1–2 full games	<code>piece_type</code> column in <code>moves_detail.csv</code> — collected by <code>ChessStatistics</code> class	Pie chart (Piece Dependency) + Bar chart, filterable by White/Black
Feature 5: Piece captures	Shows which pieces get captured most and by which color. Can be used to understand tactical patterns and which pieces are most at risk	Recorded on every capture event as one row. 100 rows = multiple games with active captures	<code>piece_type</code> , <code>captured_piece</code> , <code>color</code> columns in <code>moves_detail.csv</code> — collected by <code>ChessStatistics</code> class	Stacked bar chart (Lethality Matrix) + Summary table (count per piece, split by White/Black)

Feature 6: Capture Hesitation (sec)	Average think time before executing a capture — reveals decisiveness vs over-caution. Can be used to identify which player hesitates before taking a piece and with which piece type	One row per move with is_capture=1. 100 rows = approximately 5–10 games worth of captures	<code>move_time</code> column filtered by <code>is_capture=1</code> in <code>moves_detail.csv</code> — collected by ChessStatistics class	Bar chart (avg hesitation per piece type), filterable by White/Black
Feature 7: Think Time by Phase	Compares thinking speed in Opening (moves 1–10), Middlegame (11–30), and Endgame (31+). Can be used to see if a player speeds up or slows down as the game progresses	One row per move with move_number. 100 rows = approximately 1–2 full games	<code>move_time</code> + <code>move_number</code> columns in <code>moves_detail.csv</code> — collected by ChessStatistics class	Line graph (avg move time per game phase), filterable by White/Black

4.2 Statistical Summary Table

Values will be computed from real gameplay data using `pandas.describe()` and displayed in the dashboard.

Feature	Min	Max	Mean	Median	Std Dev
Game duration (sec)	—	—	—	—	—
Move count per game	—	—	—	—	—
Move time per turn (sec)	—	—	—	—	—
Captures per game	—	—	—	—	—

fill from real data after collecting 100+ rows

4.3 Data Recording Method

File	Contents
games_history.csv	One row per game: game_id, timestamp, winner, duration, total_moves, white_moves, black_moves, white_captures, black_captures, check_events, white_time_used, black_time_used, game_type
moves_detail.csv	One row per move: game_id, move_number, piece_type, color, to_col, to_row, move_time, is_capture, captured_piece, is_check, is_castle, is_promotion

PGN files saved separately in `/pgn_files/`, one per game, named by date and time.

4.4 Visualization Graphs

	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Win/Loss/Draw outcome	Show proportion of each game result	Pie chart	Sections: Win / Loss / Draw	% of total games
Graph 2	Game duration	Show distribution of how long games last	Histogram	Duration (min)	Number of games

Graph 3	Move count per game	Track move count trend over time across games	Line graph	Game number	Total moves
Graph 4	Piece Dependency	Compare % usage per piece type (White vs Black)	Pie chart	Pie sections: piece type	% of total moves
Graph 5	Lethality Matrix	Show which piece type captures which — attacker × victim relationship	Stacked bar chart	Attacker piece type	Number of captures, stacked by victim piece
Graph 6	Capture Hesitation	Show avg think time before capturing — reveals decisiveness per piece type	Bar chart	Piece type	Avg seconds before capture
Graph 7	Think Time by Phase	Compare move speed across Opening, Middlegame, Endgame phases	Line graph	Game phase	Avg move time (sec)

Graph categories covered:

Category	Graphs Used
Distribution	Histogram (Graph 2)
Time-series	Line graph (Graph 3, Graph 7)
Proportion	Pie chart (Graph 1, Graph 4)
Comparison	Bar chart (Graph 6)
Relationship	Stacked bar chart — Lethality Matrix (Graph 5)

Dynamic Filter: All charts support filtering via 5 toggle buttons in the chart viewer — All, White Side, Black Side, Time/Speed, Captures — allowing direct comparison of playstyle between both sides without leaving the dashboard.

5. Project Planning Timeline

Week	Week No.	Task
1	8	Proposal v2.0, repo setup, basic Pygame window and board rendering
2	9	Square class, ChessBoard with 8x8 grid, piece images and click-to-select
3	10	ChessPiece base class + all subclasses with inheritance, basic movement
4	11	Chess960Generator algorithm, full move validation per piece subclass
5	12	Special moves — Chess960 castling, en passant, pawn promotion, check, checkmate
6	13	Time controls, game state management, game over screen, PGNExporter
7	14	Statistics recording per turn, CSV export, real-time notation panel
8	15	Advanced Analytics Dashboard: Pandas-based summary table, 7 chart types with dynamic filter (White/Black/All), Capture Hesitation, Lethality Matrix, Think Time by Phase — final testing, documentation, video

Checkpoint targets:

- 50% (Week 12): Fully playable Chess960 with all rules and time controls
 - 75% (Week 13): Statistics recording, PGN export, and notation display functional
 - 100% (Week 15): Complete project with Advanced Analytics Dashboard, documentation, and video
-

6. Document Version

Date	Name	Description
14/3	Nattanan	OOP feedback: use Square class, use inheritance for pieces, separate classes by object
6/4	Phasathat	v2.0 — Changed to Chess960, redesigned OOP with Square + inheritance, fixed heatmap, added Chess960Generator, updated stats to per-turn collection

Apr 2026	Phasathat	v2.1 — Added Real-time Notation Panel, PGN Export, Advanced Analytics Dashboard (7 charts, dynamic filter, Lethality Matrix, Capture Hesitation, Think Time by Phase), updated CSV schema to match chess_statistics.py
-------------	-----------	--